Aspect-Based Sentiment Analysis on TripAdvisor Hotel Reviews

UE24CS352A Machine Learning – Mini-Project Write-up

Team: Shikshith V (PES1UG24AM400) and Sateesh (PES1UG24AM397) | **Code:** github.com/Shikshith05/ML-Miniproject (private repo, shared with faculty)

1. Problem Statement

A single star rating does not tell us why a guest liked or disliked a hotel. In this project we read the text of a TripAdvisor review and predict the **overall star rating** and the star ratings of **five aspects**: value, rooms, location, cleanliness and sleep quality (all on a 1–5 scale). We then turn the stars into **polarity**: below 3 is negative, 3 is neutral and above 3 is positive. The project is based on the Stanford CS229 report *Aspect-based Sentiment Analysis on Hotel Reviews* [1]. Our goals were to (i) compare Linear SVM, Naive Bayes and Ridge regression, (ii) check how they do against a simple "always guess 5 stars" baseline, (iii) see whether training on an aspect's own ratings works better than training only on overall ratings, and (iv) build a small live demo. The paper also uses Service, Business Service and Check-in, but our dataset does not have them, so we only predict the overall rating and five aspects.

2. Dataset Details

We used the Hugging Face dataset `jniimi/tripadvisor-review-rating` [2], a cleaned English-only TripAdvisor dataset derived from Li, Ott and Cardie (2013). Each row has the review title and body (column `review`), an `overall` rating, and ratings for cleanliness, value, location, rooms and sleep quality. The full file has about 201,000 reviews, so to run it on a laptop we randomly sampled **30,000** (seed 42). We dropped reviews with 20 characters or fewer and treated ratings outside 1–5 as missing, so no model trains on a rating that does not exist. After this, all six ratings were present for every review. We split the data into **20,000 training and 10,000 test** reviews (2/3 and 1/3, like the paper). The ratings are **heavily skewed**: 5 stars is the most common rating for every target, and always predicting 5 stars already gives 41–64% accuracy (63.8% for location). So accuracy alone can mislead, and we also report macro-F1, within-1-star accuracy and the baseline.

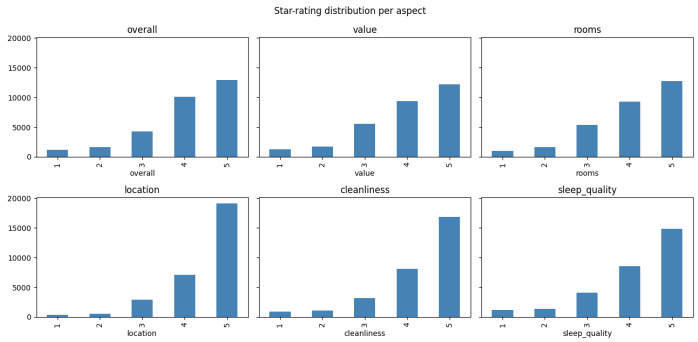


Figure 1. Star-rating distribution for each target (30,000 sampled reviews).

3. Approach

Features. We turn each review into counts of word 1–3-grams (`CountVectorizer`) with `min_df=2`, `max_df=20%` and at most 300,000 features. The vocabulary is learned only from the training reviews. We kept stop words so that phrases like "not good" stay intact. We did not use TF-IDF: the paper says sentiment words such as "great" and "dirty" appear in many reviews and TF-IDF would reduce their weight. We took this choice from the paper and did not test TF-IDF ourselves.

Models. For each of the six targets we trained a Linear SVM (`C=0.1`, balanced class weights so the rare 1–2 star classes are not ignored), Multinomial Naive Bayes (`alpha=0.5`) and Ridge regression (`alpha=5`) with the output rounded to the nearest star from 1 to 5. That makes 18 models. The settings were fixed and we did not tune them.

Metrics. Exact star accuracy, within-1-star accuracy, macro-F1 and polarity accuracy, each compared with the majority-class baseline.

Main experiment. We trained a Linear SVM only on the overall ratings and used it to predict each aspect, then compared it with the SVM trained on that aspect's own ratings.

4. Implementation Overview

All the code is in one notebook, `absa_hotel_reviews.ipynb` (Steps 0–12). It loads and cleans the data, plots the ratings, splits the data, builds the features, trains the 18 models and evaluates them against the baseline. It also has the confusion matrix, the aspect-vs-overall experiment, the top phrases for each aspect, the saved models (joblib) and a short summary cell. The function `predict_review(text)` gives stars and polarity for all six targets and is what we use for the live demo. Tables and plots are saved in the `outputs/` folder, and every number in this report comes from those saved tables. Setup and run steps are in the README.

5. Results

	Star accuracy				Polarity accuracy			
Target	**Base**	**SVM**	**NB**	**Ridge**	**Base**	**SVM**	**NB**	**Ridge**
Overall	0.432	0.618	**0.651**	0.560	0.764	0.839	**0.854**	0.824
Value	0.409	0.485	**0.525**	0.445	0.724	0.756	**0.775**	0.757
Rooms	0.425	0.536	**0.560**	0.499	0.732	0.779	**0.787**	0.779
Location	0.638	0.626	**0.646**	0.557	0.876	0.861	**0.862**	0.861
Cleanliness	0.560	0.589	**0.626**	0.548	0.826	0.845	**0.850**	0.839
Sleep quality	0.496	0.547	**0.578**	0.496	0.776	0.799	**0.801**	0.792

Table 1. Test results on 10,000 reviews (Base = always predict 5 stars, SVM = Linear SVM, NB = Naive Bayes). Best model per column in bold.

Model comparison. Naive Bayes had the highest star accuracy on every target (for example 0.651 on overall, against 0.618 for the SVM and 0.560 for Ridge). The balanced SVM gives up some accuracy to treat rare classes better, and it had a higher macro-F1 than Naive Bayes for rooms (0.463 vs 0.455) and location (0.312 vs 0.285). Most predictions are close to the true rating: within-1-star accuracy is 0.89–0.97 for all models, and the confusion matrix (Figure 2) shows that the mistakes are mostly between neighbouring ratings like 4 and 5.

Baseline. The best model beats the baseline in star accuracy by 0.219 (overall), 0.115 (value), 0.135 (rooms), 0.081 (sleep quality) and 0.067 (cleanliness), so for these it really learns from the text. For **location** the gain is only 0.008 (the SVM is 0.012 below the baseline), and the polarity baseline (0.876) is higher than every model (0.861–0.862). So we cannot say our models learned location sentiment beyond predicting the most common rating.

Aspect-trained vs overall-trained SVM (star accuracy on each aspect's own ratings):

Target	Aspect-trained	Overall-trained	Difference
Value	0.485	0.502	-0.017
Rooms	0.536	0.545	-0.009
Location	0.626	0.494	+0.132
Cleanliness	0.589	0.556	+0.033
Sleep quality	0.547	0.534	+0.014

Table 2. Training on an aspect's own ratings helps for location, cleanliness and sleep quality, but not for value and rooms. The location gain is mostly because the overall-trained model scores below the baseline there (0.494).

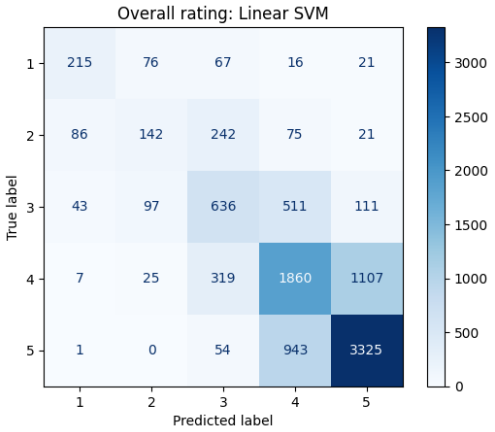


Figure 2. Confusion matrix for the overall rating, Linear SVM (rows = true stars, columns = predicted).

6. Conclusions

Simple 1–3-gram counts with linear models predict ratings from review text much better than guessing for overall, value, rooms, cleanliness and sleep quality, and the predictions are usually within one star (95.6% for the overall SVM). Naive Bayes is our most accurate model. We kept the balanced SVM for comparison because it handles rare low ratings better. Training on an aspect's own ratings helps most when the aspect has its own words, like cleanliness ("spotless", "dirty").

Limitations. Each model reads the whole review, so it cannot tell which sentence is about which aspect. In our demo, a review that said the location was "far from everything" still got a positive location score. Some of the top features are general sentiment words (for example "classy" for sleep quality), the 300,000-feature limit was reached, ratings are skewed toward 5 stars, and our dataset has five aspects instead of the paper's seven, so our numbers cannot be compared directly with the paper's. We also did not tune hyperparameters or use cross-validation.

Future work. Assign each sentence to an aspect before scoring it, try class rebalancing or ordinal regression, and try transformer models such as BERT.

References. [1] Y. Yu, *Aspect-based Sentiment Analysis on Hotel Reviews*, CS229 project report, Stanford University, 2016. [2] Hugging Face dataset jniimi/tripadvisor-review-rating (huggingface.co/datasets/jniimi/tripadvisor-review-rating), derived from Li, Ott and Cardie (2013).